

Which Model When

A working reference for classical ML model selection. Left side is for fast interview recall, right side for real project calls. No deep learning, no LLMs.

01 First, what is tabular data?

Rows are records, columns are features. A spreadsheet. Most classical ML lives here. This one loan table is reused across every model below so you can see what each is for.

AGE	INCOME	LOAN AMT	CREDIT SCORE	EMPLOYED	APPROVED
28	45,000	15,000	680	Yes	Yes
45	90,000	50,000	720	Yes	Yes
33	30,000	40,000	590	No	No
52	120,000	20,000	810	Yes	Yes

02 The 30-second decision

- 1 Is the target a **label or a number**? Label to classification, number to regression. No target to clustering or dimensionality reduction.
- 2 Need to **explain every prediction** to a stakeholder? Start with Linear/Logistic Regression or a single Decision Tree.
- 3 Want **best accuracy on tabular data** and can trade interpretability? Go straight to Gradient Boosting (XGBoost / LightGBM).
- 4 Dataset **small or noisy**, features many? Regularized linear models or SVM.
- 5 Only when tabular boosting underperforms do you reach for anything heavier.

Linear / Logistic Regression

baseline · interpretable · fast

USE Linear relationships, need coefficients you can explain, want a baseline before anything fancy.

SKIP Strong non-linear patterns, heavy feature interactions.

WATCH Scale features, check multicollinearity, regularize (L1/L2) when features outnumber clean signal.

EX Predict **Approved (Yes/No)** from the loan table, with a coefficient per feature you can explain. Swap target to a number: predict **Loan Amount offered**.

Decision Tree

interpretable · non-linear · unstable

USE Need a human-readable rule set, mixed feature types, non-linear splits, no scaling needed.

SKIP You need stable, high accuracy. A single tree overfits easily.

WATCH Prune or cap depth. Tiny data changes reshape the whole tree.

EX Same loan table, output is a printable rule: **if credit score < 600 and employed = No then reject**.

Random Forest

robust · low-tuning · tabular

USE Strong default for tabular. Handles non-linearity and noise, minimal tuning, gives feature importance.

SKIP Very large data where boosting wins, or when you need the single best score.

WATCH Larger and slower than boosting at similar accuracy. Less interpretable than one tree.

EX Identical loan table, hundreds of trees vote on **Approved**. Steadier than one tree, almost no tuning.

Gradient Boosting (XGBoost / LightGBM)

top tabular accuracy • tunable

USE The go-to for winning tabular performance, structured data, competitions, most production tabular tasks.

SKIP Tiny datasets, or when full interpretability is mandatory.

WATCH Sensitive to hyperparameters. Can overfit without early stopping and learning-rate control.

EX Same loan table but **200,000 rows and 40 columns**. Squeezes out the highest accuracy on this structured data.

Support Vector Machine (SVM)

small data • high-dim • margins

USE Small-to-medium datasets, high-dimensional spaces (text, genomics), clear margins. Kernels for non-linear.

SKIP Large datasets (slow to train), or when you need probability outputs cheaply.

WATCH Must scale features. Kernel + C + gamma tuning matters a lot.

EX **80 applicants, 300 columns** (every transaction category, device signal). Few rows, many features. SVM handles that shape.

k-Nearest Neighbors (kNN)

no training • lazy • intuitive

USE Small datasets, low dimensions, quick baseline, recommendation-style similarity.

SKIP Large or high-dimensional data. Prediction is slow and distances break down.

WATCH Scale features, choose k carefully, suffers from the curse of dimensionality.

EX New applicant arrives. Find the **5 most similar past applicants** by age/income/score, predict the majority outcome.

Naive Bayes

text · fast · probabilistic

USE Text classification, spam filtering, high-dimensional sparse features, very fast baseline.

SKIP Features are strongly correlated, or you need calibrated probabilities.

WATCH The independence assumption is usually false but often works anyway.

EX Leave loans: table of emails, columns are **word counts**. Predict **spam / not spam**.

Regularized Linear (Ridge / Lasso / ElasticNet)

high-dim · feature selection

USE Many features, risk of overfitting. Lasso zeros out weak features; Ridge shrinks; ElasticNet blends.

SKIP Clearly non-linear relationships that no feature engineering fixes.

WATCH Tune the regularization strength via cross-validation, always scale first.

EX Loan table blown up to **500 columns**. Lasso zeros out the useless ones and keeps the few that predict **Approved**.

K-Means

clustering · fast · spherical

USE You roughly know the number of clusters, groups are compact and similar in size.

SKIP Irregular shapes, varying density, or unknown cluster count.

WATCH Must set k, scale features, sensitive to initialization and outliers.

EX Drop the **Approved** column. Group customers by age and income into **3 segments**, no target needed.

DBSCAN

density · finds outliers · no k

USE Arbitrary cluster shapes, unknown cluster count, want automatic outlier detection.

SKIP Clusters of very different densities, high dimensions.

WATCH eps and min_samples are fiddly. Distance meaning degrades in high dimensions.

EX Same customers, but clusters are odd-shaped and a few applicants sit far off. DBSCAN groups them and **flags the outliers**.

Hierarchical (Agglomerative)

dendrogram · nested groups

USE Want a cluster hierarchy or dendrogram, small datasets, don't want to preset k.

SKIP Large datasets, it scales poorly.

WATCH Linkage choice (ward, average, complete) changes results significantly.

EX Build a **dendrogram** of customers, from every-person-alone up to one big group, and cut it where the segments look right.

PCA

dim-reduction · linear · preprocessing

USE Reduce dimensions, remove correlation, speed up models, visualize variance structure.

SKIP You need interpretable features, or structure is non-linear (use t-SNE / UMAP to visualize).

WATCH Scale first. Components are combinations, not original features.

EX **50 correlated financial columns** compressed into ~5 combined dimensions, so the other models run faster.

05 Fast lookup by situation

SITUATION	FIRST REACH FOR	WHY
Tabular, want best accuracy	XGBoost / LightGBM	Consistently strongest on structured data
Need to explain to stakeholders	Logistic Reg / single Tree	Coefficients or readable rules
Quick strong baseline	Random Forest	Robust with almost no tuning
Text classification	Naive Bayes / Linear SVM	Fast on sparse high-dim features
Small, high-dimensional data	SVM	Works well when samples are few
Many features, overfitting	Lasso / ElasticNet	Shrinks and selects features
Group data, know cluster count	K-Means	Fast and simple
Group data, odd shapes / outliers	DBSCAN	No k needed, flags noise
Too many features to model	PCA	Compress before modeling
Tiny dataset, quick check	kNN	No training, intuitive

06 When to scale or normalize

Scaling matters when a model measures **distance** or uses **gradients** / **regularization**. It does not matter for tree-based models, which split one feature at a time. Fit the scaler on the **training fold only**.

MODEL	SCALE?	WHY
SVM	Yes	Uses distances and margins
kNN	Yes	Pure distance; big-range features dominate
K-Means	Yes	Distance to centroids
PCA	Yes	Variance-driven; scale sets the axes
Linear/Logistic + L1/L2	Yes	Penalty is unfair if scales differ
Neural nets	Yes	Gradients converge faster when scaled
Decision Tree	No	Splits one feature at a time
Random Forest	No	Tree-based; scale-invariant
Gradient Boosting	No	Tree-based; scale-invariant
Naive Bayes	No	Works on probabilities / counts

Standardization (Z-score)

mean 0 • std 1 • default choice

USE Roughly bell-shaped data, and for SVM, linear models, PCA. The safe default.

EX Income and Age put on the same scale so neither drowns the other.

Normalization (Min-Max)

range 0 to 1 • bounded

USE You need a fixed range, non-Gaussian data, or image pixels. Sensitive to outliers.

EX Credit Score (300-850) squeezed to 0-1 alongside other bounded features.

Rule: use **RobustScaler** when outliers are present (it uses medians). Fit any scaler on train only, then apply to test, never fit on the full dataset.

07 Evaluation metrics

Pick the metric **before** comparing models. The wrong metric picks the wrong model, especially on imbalanced data.

Classification

METRIC	USE WHEN	CAREFUL
Accuracy	Classes are balanced	Lies on imbalance (99% "no fraud")
Precision	False positives are costly	Ignores missed positives
Recall	False negatives are costly (fraud, disease)	Ignores false alarms
F1	Need balance of precision & recall	Hides which one is weak
ROC-AUC	Rank quality, roughly balanced	Over-optimistic on heavy imbalance
PR-AUC	Imbalanced data, positive class matters	The right AUC for rare events

Regression

METRIC	USE WHEN	CAREFUL
MAE	Want error in real units, robust to outliers	Treats all errors linearly
MSE / RMSE	Large errors should hurt more	Outliers dominate the score
R²	How much variance is explained	Can rise just by adding features
MAPE	Want error as a percentage	Breaks near zero values

Clustering

METRIC	USE WHEN	CAREFUL
Silhouette	No true labels; measures separation	Assumes convex clusters
Inertia / Elbow	Choosing k for K-Means	Always drops; judge the bend
Adjusted Rand	You do have true labels to compare	Needs ground truth

Fraud example: 99.5% of transactions are legit. A model predicting "all legit" scores 99.5% accuracy and catches zero fraud. Use **recall** and **PR-AUC** instead.

BIAS/VAR Linear = high bias, low variance. Deep tree / kNN(low k) = low bias, high variance. Ensembles cut variance.

BAGGING Random Forest: parallel trees on bootstrapped samples, reduces variance.

BOOSTING Sequential trees fixing prior errors, reduces bias. XGBoost/LightGBM.

SCALING Needed for SVM, kNN, K-Means, PCA, linear-with-regularization. Not for tree-based models.

IMBALANCE Class weights, resampling (SMOTE), and judge with precision/recall/F1 or PR-AUC, not accuracy.

L1 VS L2 L1 (Lasso) drives weights to zero (selection). L2 (Ridge) shrinks smoothly, keeps all.

no free lunch

baseline first

CV everything

leakage kills

metric $\neq$ accuracy

Most common real-world mistake: data leakage. Fit scalers, encoders, and imputers on the training fold only, inside cross-validation, never on the full dataset before splitting.